Using Codex to Build a YouTube Transcript Workflow

Table of Contents

- What This Tutorial Is For
- Prerequisites
- 1. Introduction and Tutorial Overview
- 2. Starting a New Transcript Project
- 3. Planning the Project with AI Assistance
- 4. Downloading and Processing YouTube Transcripts
- 5. Executing the Development and Review Workflow
- Recap

What This Tutorial Is For

This tutorial walks through a practical workflow for using Codex to build a small transcript-processing project around YouTube videos.

It is for technical users who are comfortable working on a Mac or similar system, organizing project folders, and using AI tools to help plan and build software.

By the end, you will have a clear model for how to:

- start a new transcript project
- organize the project files
- use AI as a planning partner
- download and work with YouTube transcripts
- move through coding, testing, review, and refinement

Why this workflow matters: instead of treating AI as a one-shot code generator, this process uses it as a co-thinker for product definition, design, sprint planning, implementation, and review. That makes the work easier to structure and easier to improve in iterations.

Back to top

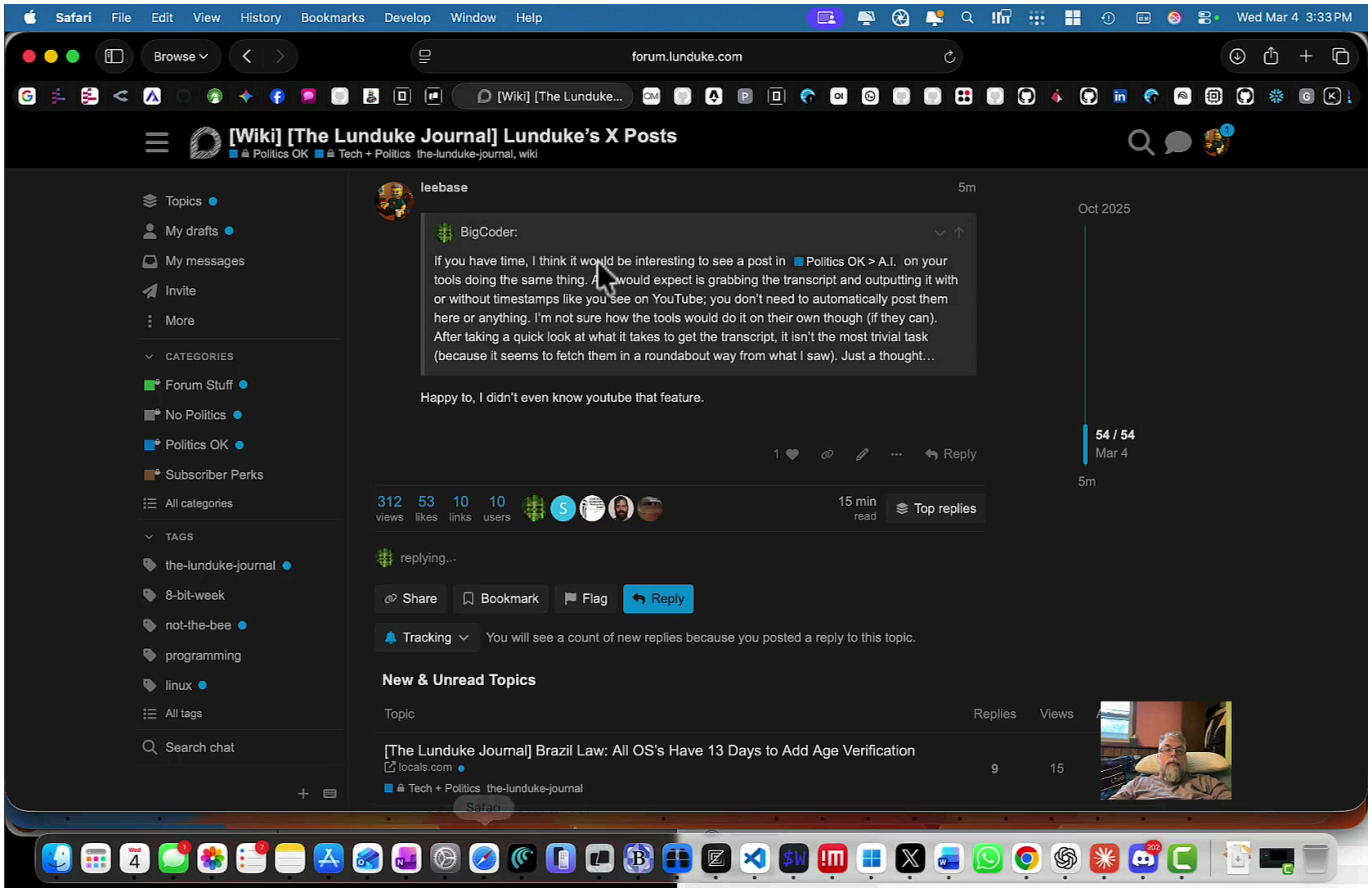
Prerequisites

Before you begin, make sure you have:

- basic familiarity with using a Mac or similar computer environment
- an understanding of project folder organization
- basic knowledge of AI tools and their use in content creation
- access to the video or transcript for reference
- access to the AI tools used in the workflow, including GPT 5.3 codecs through ChatGPT Plus, if you want to mirror the demonstrated setup

Back to top

1. Introduction and Tutorial Overview



Introduction to the tutorial covering codecs, project setup, AI planning, and transcript processing.

This workflow combines project setup, transcript collection, AI planning, coding, and review into one repeatable process.

What this tutorial covers

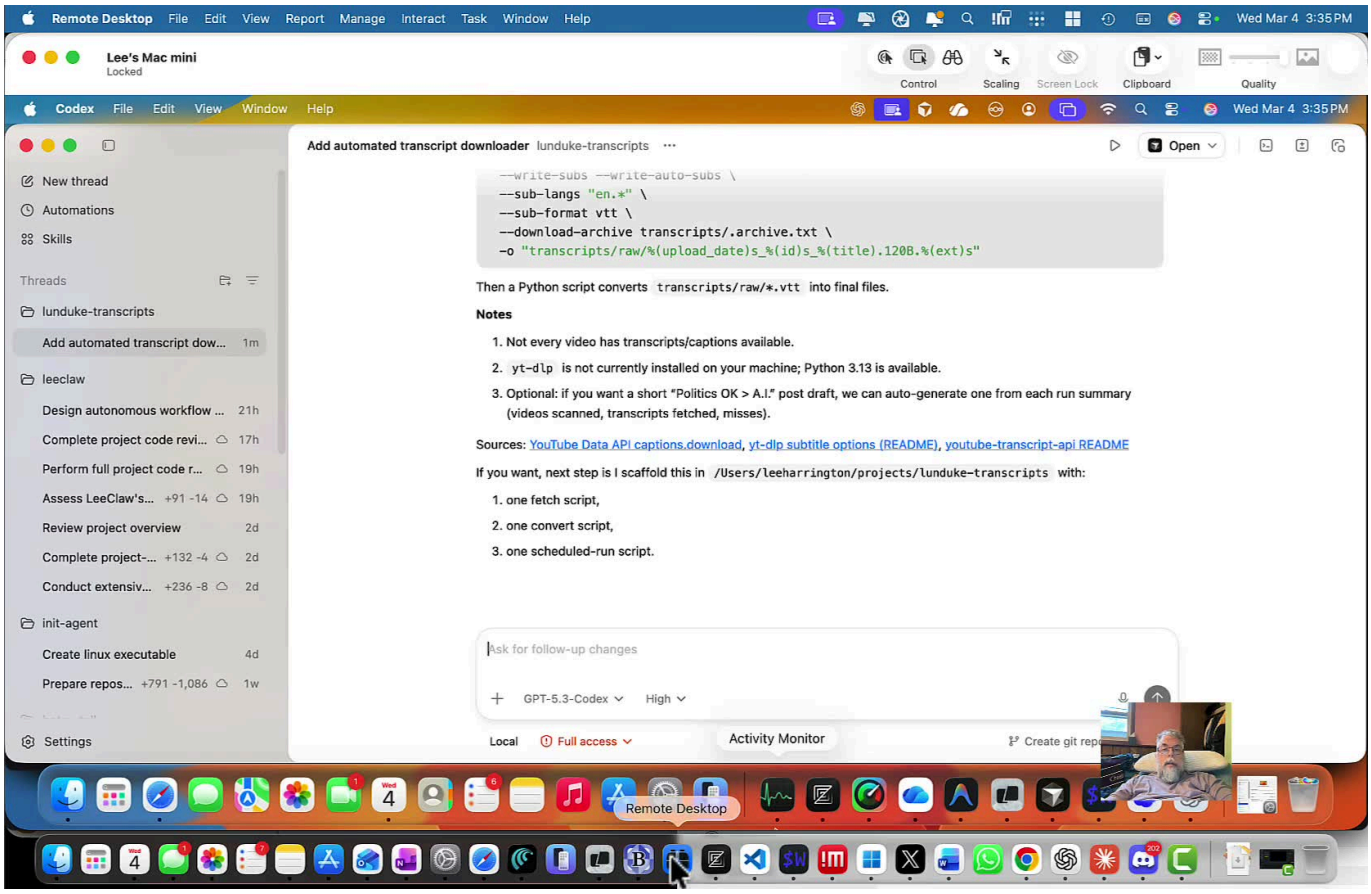
You will work through a project flow that includes:

- creating a new transcript project
- organizing files in one place
- planning the work with AI
- fetching YouTube transcripts
- building and refining the supporting utility

What you will have by the end

By the end of the tutorial, you should understand how to:

- start and organize a transcript-processing project
- define goals with AI assistance
- design a simple implementation approach
- fetch and format YouTube transcript data
- run a structured coding and review loop



Explanation of why the AI-assisted workflow matters for planning and coding.

Why this workflow matters

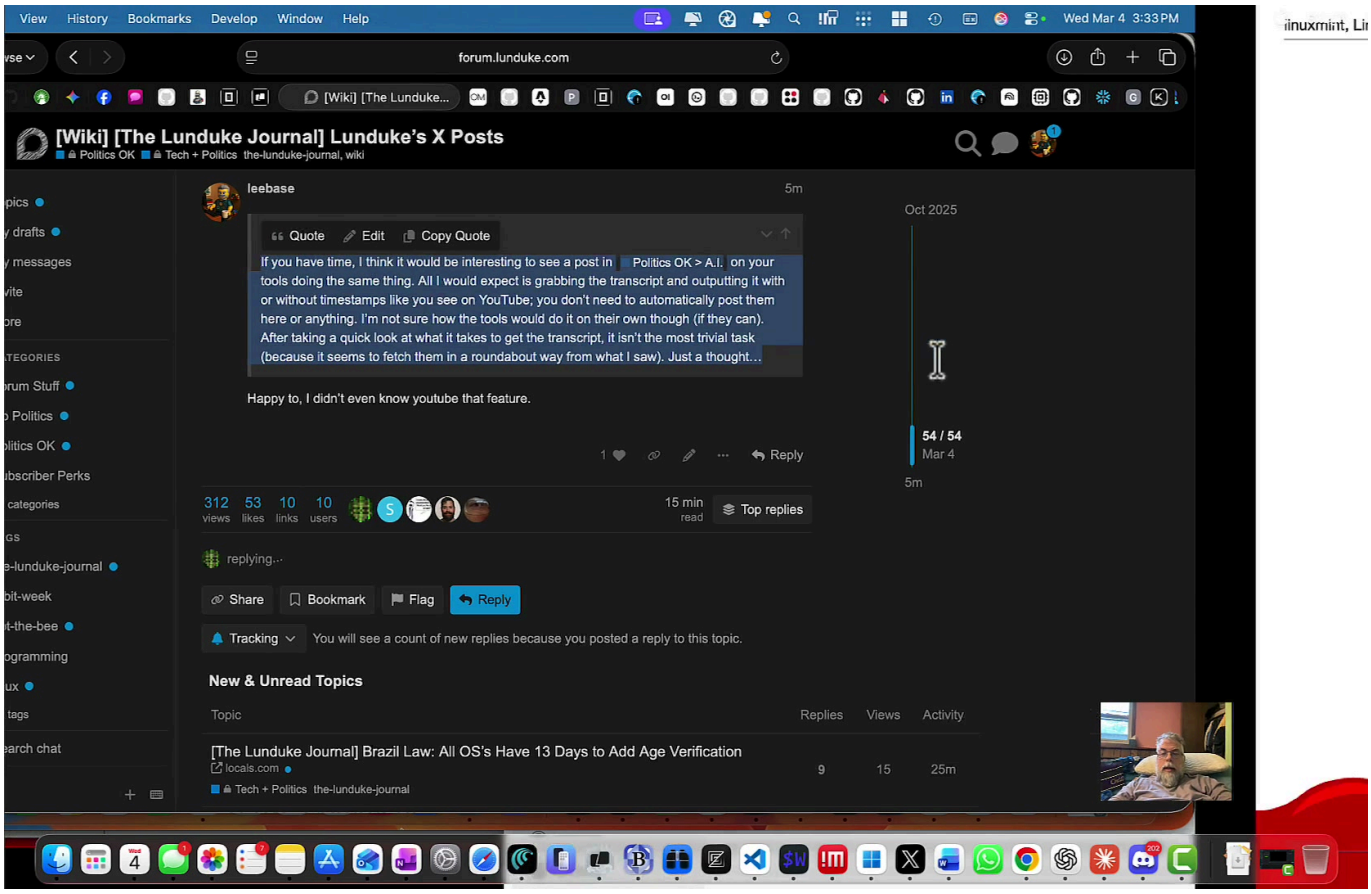
A key idea in this process is to use AI as a co-mind or co-thinker, not just as a tool that spits out code. The workflow starts with defining what needs to be built, then turns that into a design, then into sprints, then into code and review.

That structure helps you:

- clarify the outcome before implementation
- break work into testable chunks
- recover more easily if work is interrupted
- review quality before calling the project done

[Back to top](#)

2. Starting a New Transcript Project



Creating a new project folder named 'Lunduke Transcript'.

The first concrete step is to create a folder for the project. In the demonstrated workflow, the project folder is named **Lunduke Transcript**.

Create a project folder

Create a new folder that will hold everything related to the transcript project, such as:

- transcript files
- generated outputs
- project notes
- code for the utility

Using a dedicated folder keeps the work contained and makes the next AI-assisted steps easier to manage.

If you want to follow the example closely, use:

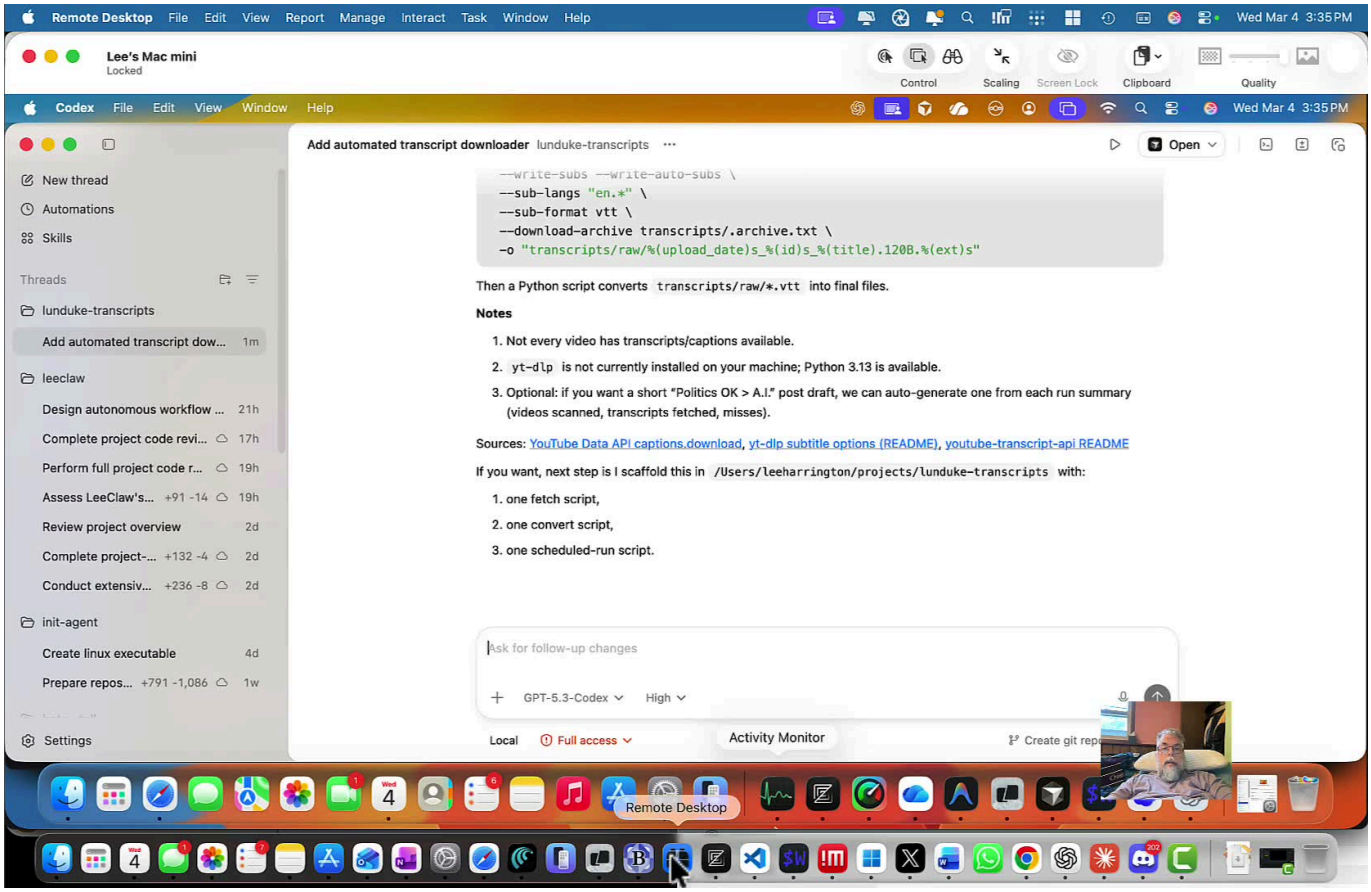
Lunduke Transcript

You do not need the exact same name, but keeping one clearly named project directory is part of the workflow.

Back to top

3. Planning the Project with AI Assistance

This section is the core of the process. Before coding, use AI to help define what you are trying to build.



Engaging AI as a co-thinking partner to define project goals and plan workflow.

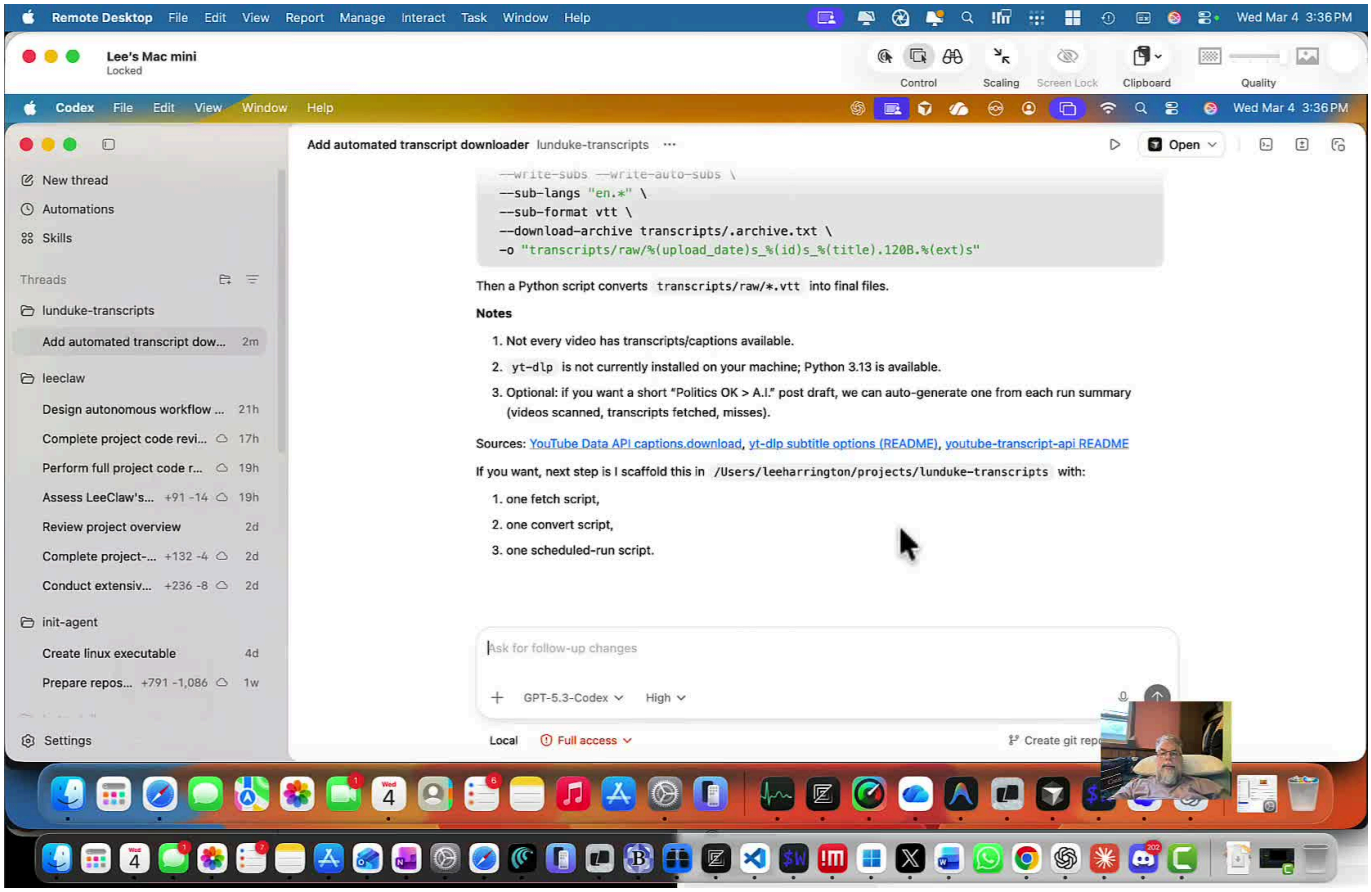
Engage AI as a co-thinking partner

Start the project by interacting with AI as a planning partner. In the demonstrated workflow, GPT 5.3 codecs is used through ChatGPT Plus.

The practical idea is simple: begin by talking through the project before asking for implementation.

That means using AI to help with:

- project framing
- product definition
- design choices
- sprint planning



Defining clear product goals with AI assistance.

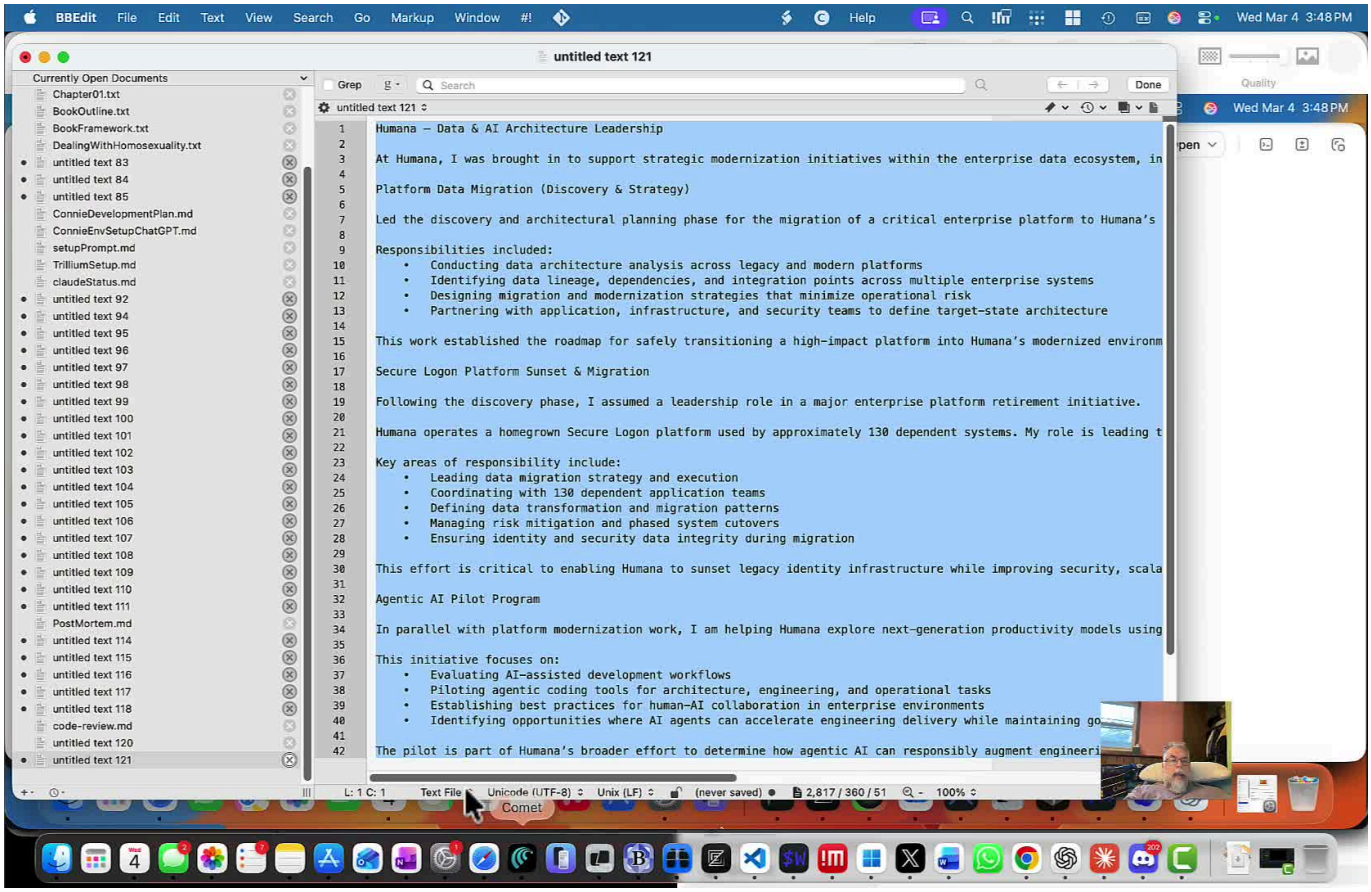
Define the product goals

Define exactly what the project should do.

In the demonstrated example, the goals include things like:

- fetching transcripts for new videos
- running the app on demand
- running automatically on a schedule
- avoiding duplicate work by recognizing videos that were already processed

This is an important step because it gives the rest of the workflow something concrete to optimize for.



Collaborating with AI to design architecture and select technology stack.

Create the project design

After the goals are clear, move into design.

The demonstrated design discussion includes a stack centered on:

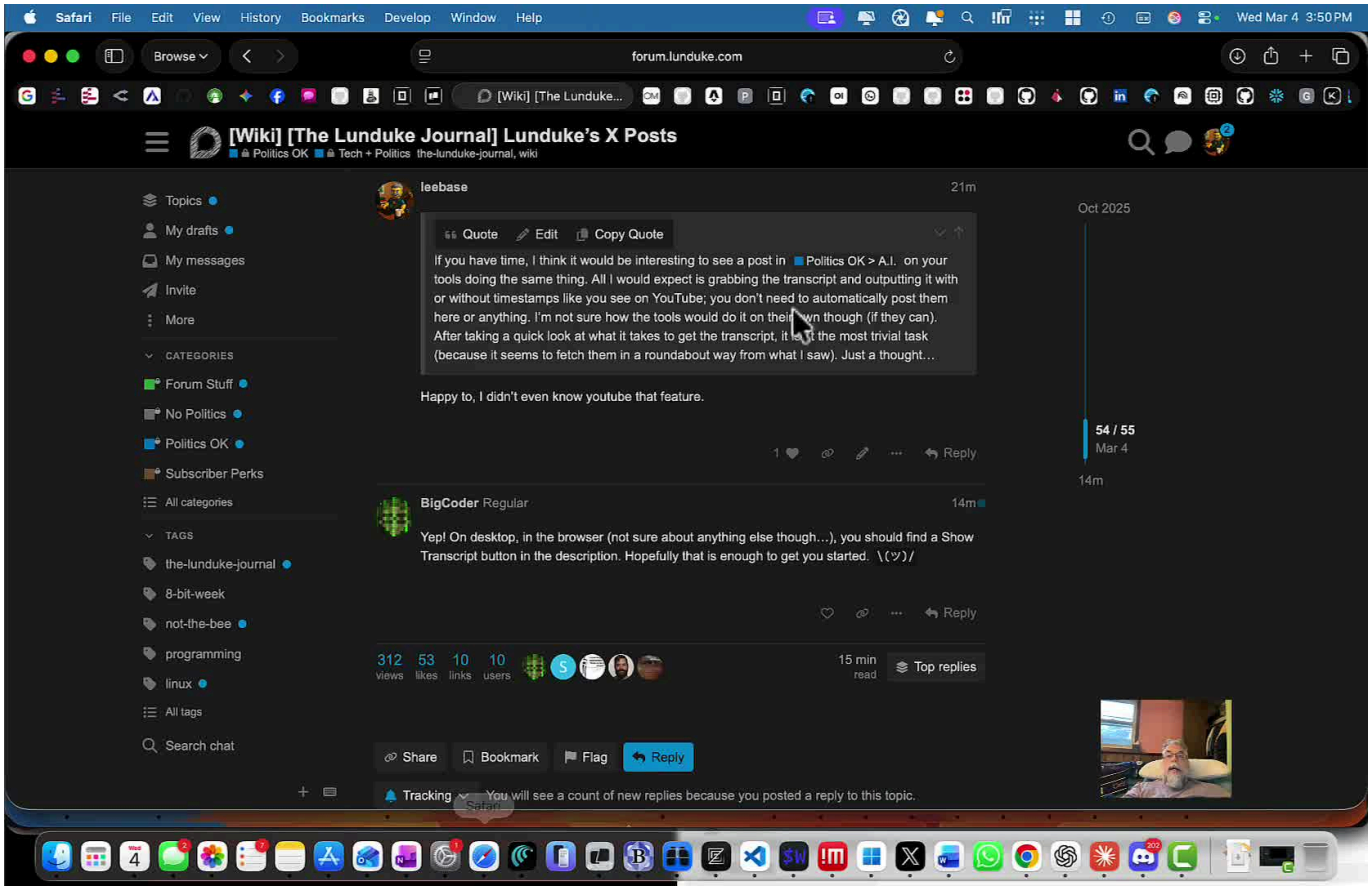
- Python
- SQLite
- data modeling and validation tools
- YouTube-related processing components

The transcript references "pedantic," but the intended tool name is not fully reliable from the source audio, so it is safest to treat this part generally as a data validation choice rather than lock in a specific package name.

At this stage, work with AI to outline:

- the main components of the app
- how transcript data will be stored
- how duplicates will be tracked

- how output files will be created



Developing a sprint plan with testable steps and checkpoints.

Develop a sprint plan

Once the design is ready, ask AI to create a sprint plan.

The demonstrated approach breaks the project into small, testable pieces with checkpoints. That matters because it gives you:

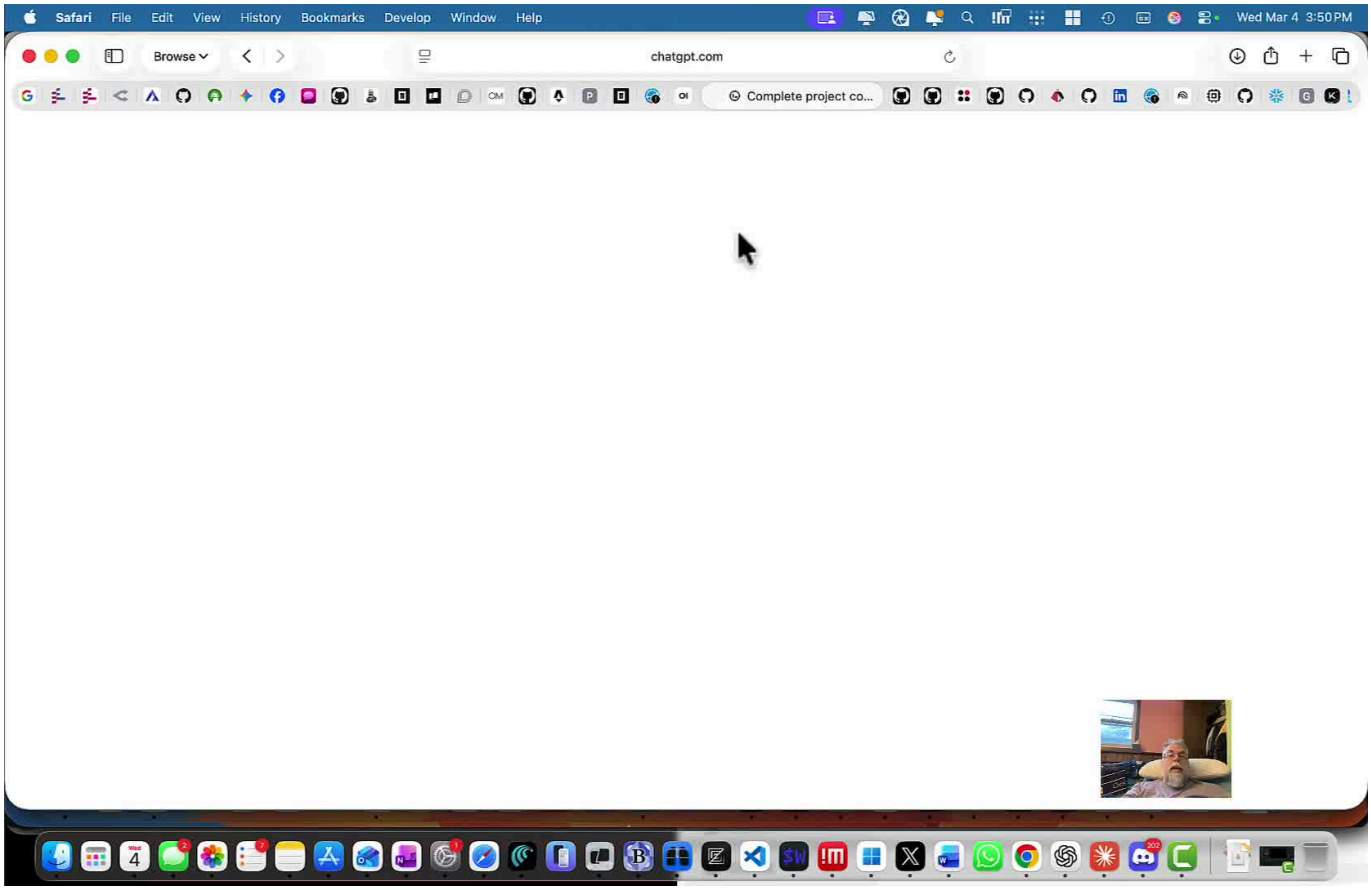
- stopping points
- testing points
- a record of what has already been completed

The sprint interface shown in the workflow also uses checkboxes so progress can be tracked step by step. This is especially useful if you need to restart or continue later.

Back to top

4. Downloading and Processing YouTube Transcripts

Once the project plan exists, the next part is getting transcript data and shaping the output.



Downloading YouTube video transcripts using AI-assisted app.

Download YouTube video transcripts

Use the app to fetch transcripts from YouTube videos.

In the demonstrated flow, the app is run against a specific YouTube channel, and later against a time range such as the **past week**.

The demonstrated workflow also shows that the app can be adjusted and rerun after feedback. For example, after reviewing results, the process is run again with updated output preferences.

Depending on how closely you are following the demonstration, this may also require entering an **OpenAI** or **OpenRouter** key before running the app.

Organize transcript files

This step is text-only because the selected frames do not clearly show the file naming details.

As you refine the output, organize transcript files so they are easier to read and reuse. The demonstrated workflow includes two important output improvements:

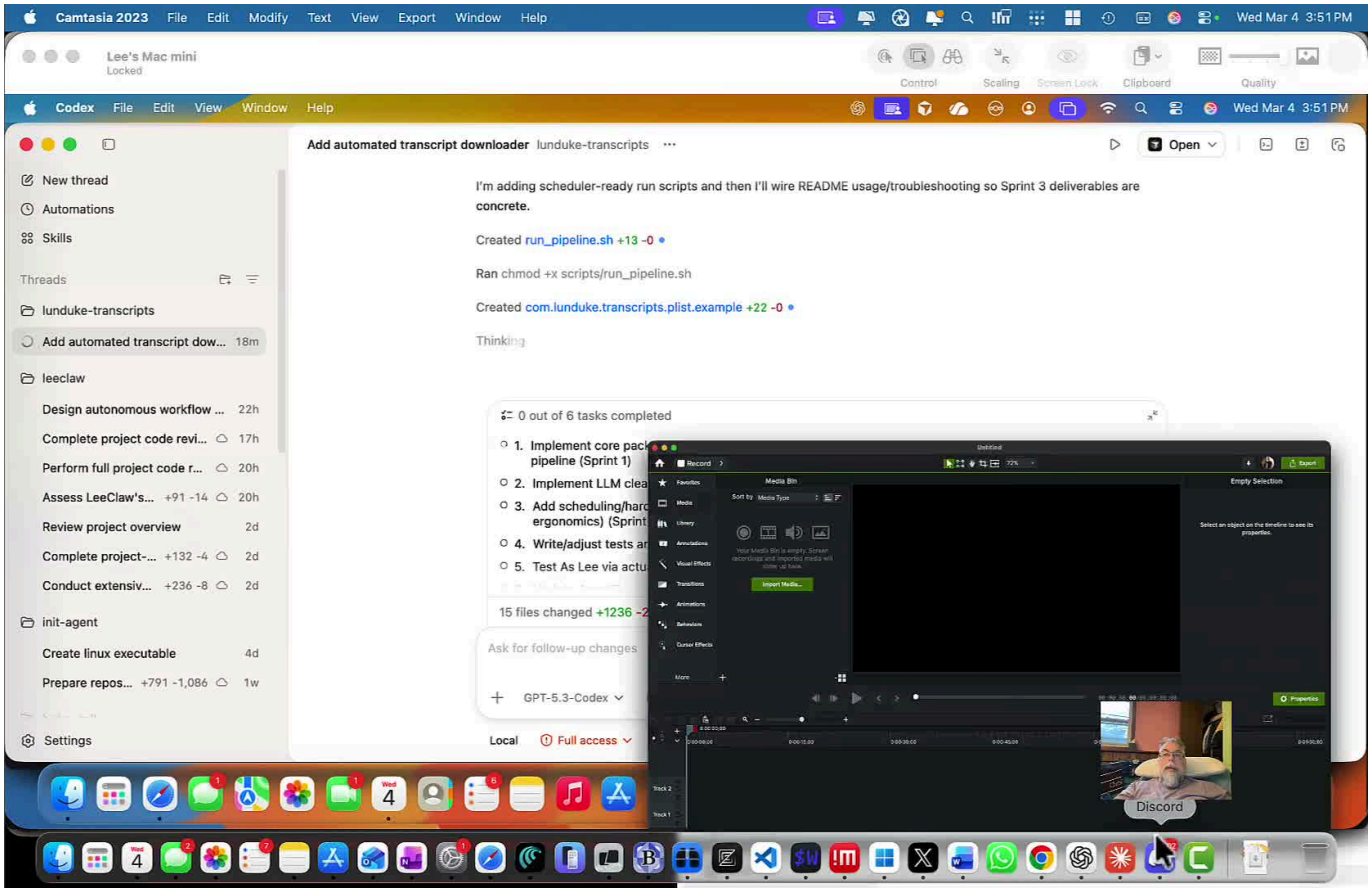
- naming files with the video title instead of an unreadable code
- placing transcript timestamps at the paragraph level, specifically at the end of each paragraph

Those two choices make the output much more usable for later writing, review, or transformation into article-style text.

Back to top

5. Executing the Development and Review Workflow

This section covers the build loop: implement, test, review, fix, and run again.



Executing coding sprints with AI coding, testing, and bug fixing.

Execute coding sprints

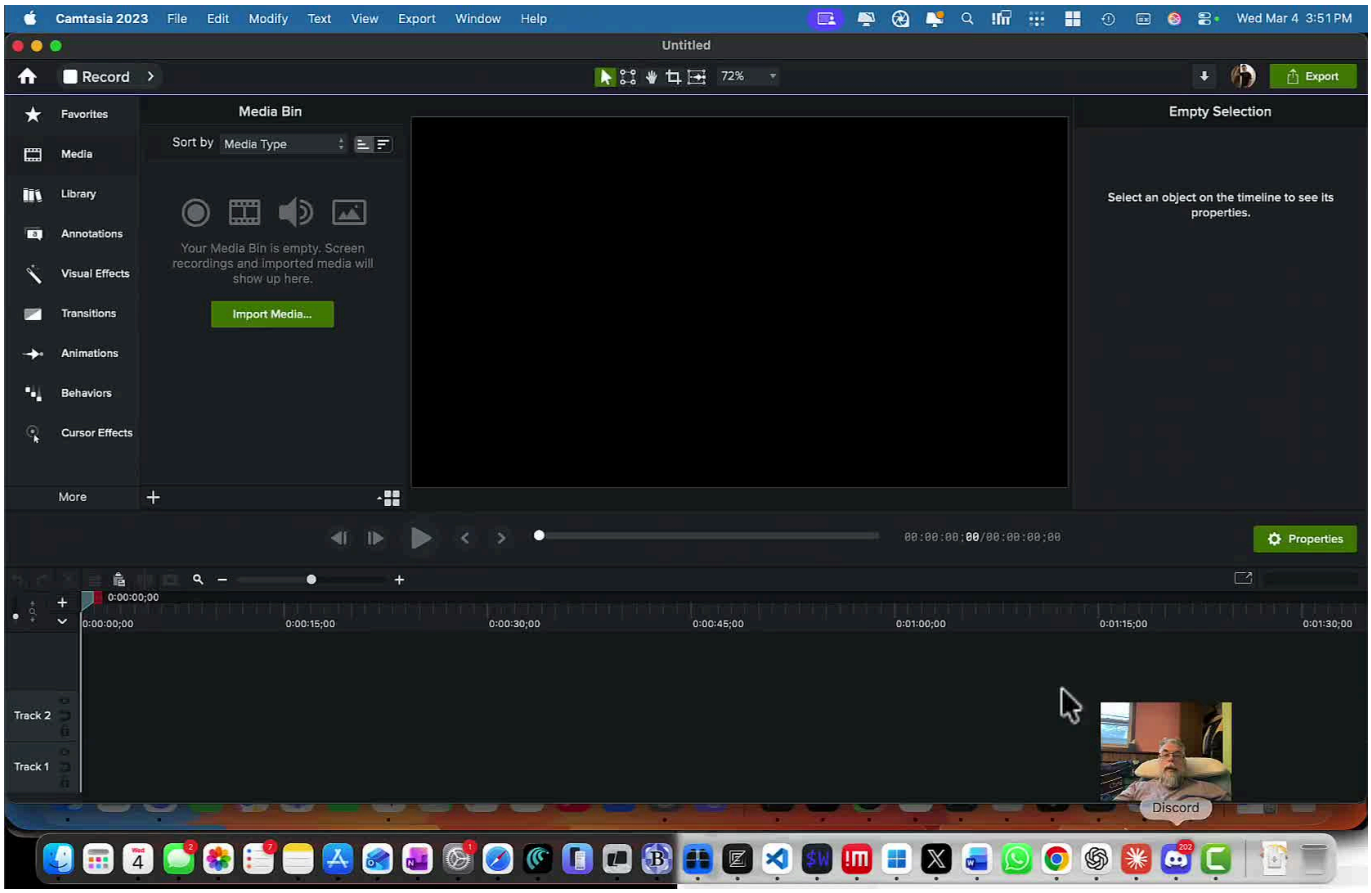
Once the sprint plan is ready, tell the AI agent to execute the sprints according to the planned flow.

The demonstrated pattern is:

1. code
2. test
3. fix bugs
4. test again as the end user

That final part matters. The workflow is not only about whether the code runs, but whether the result works from the user's point of view.

The demonstrated run also shows the agent completing work on its own and committing changes to git.



Performing comprehensive AI code review covering multiple quality aspects.

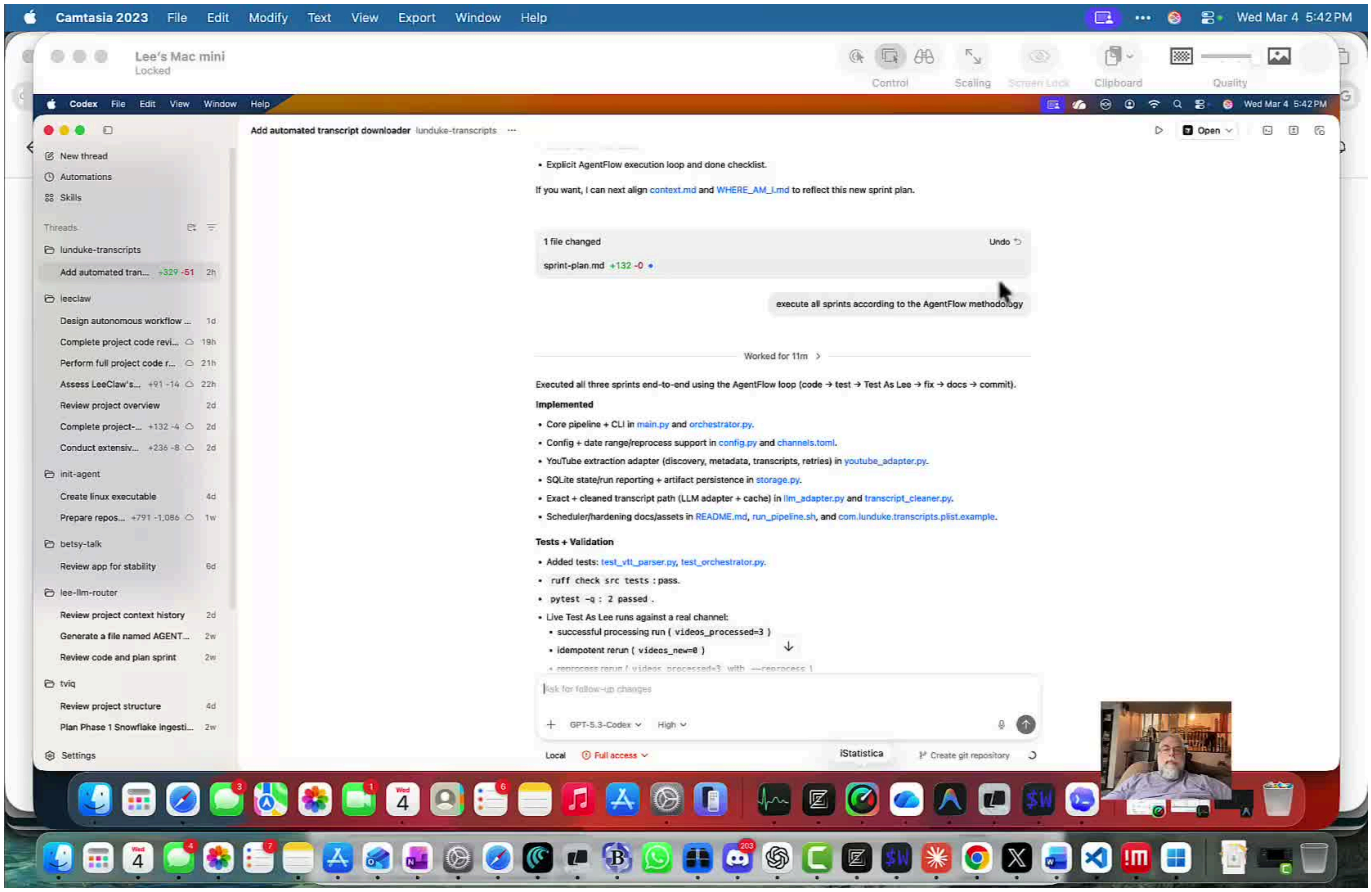
Perform code review

After implementation, run a structured code review.

In the demonstrated workflow, the review is specific rather than vague. It covers areas such as:

- formatting
- linting
- type checks
- tests
- dependency vulnerability checks
- security review
- trust boundaries
- injection risks
- correctness and reliability
- performance and scalability
- maintainability and design quality
- tests and documentation
- an executive summary or overall health view

This is a good model to follow: define what “review” means in advance instead of asking for a generic opinion.



Iterating coding and review cycles until code stability and goal achievement.

Iterate coding and review

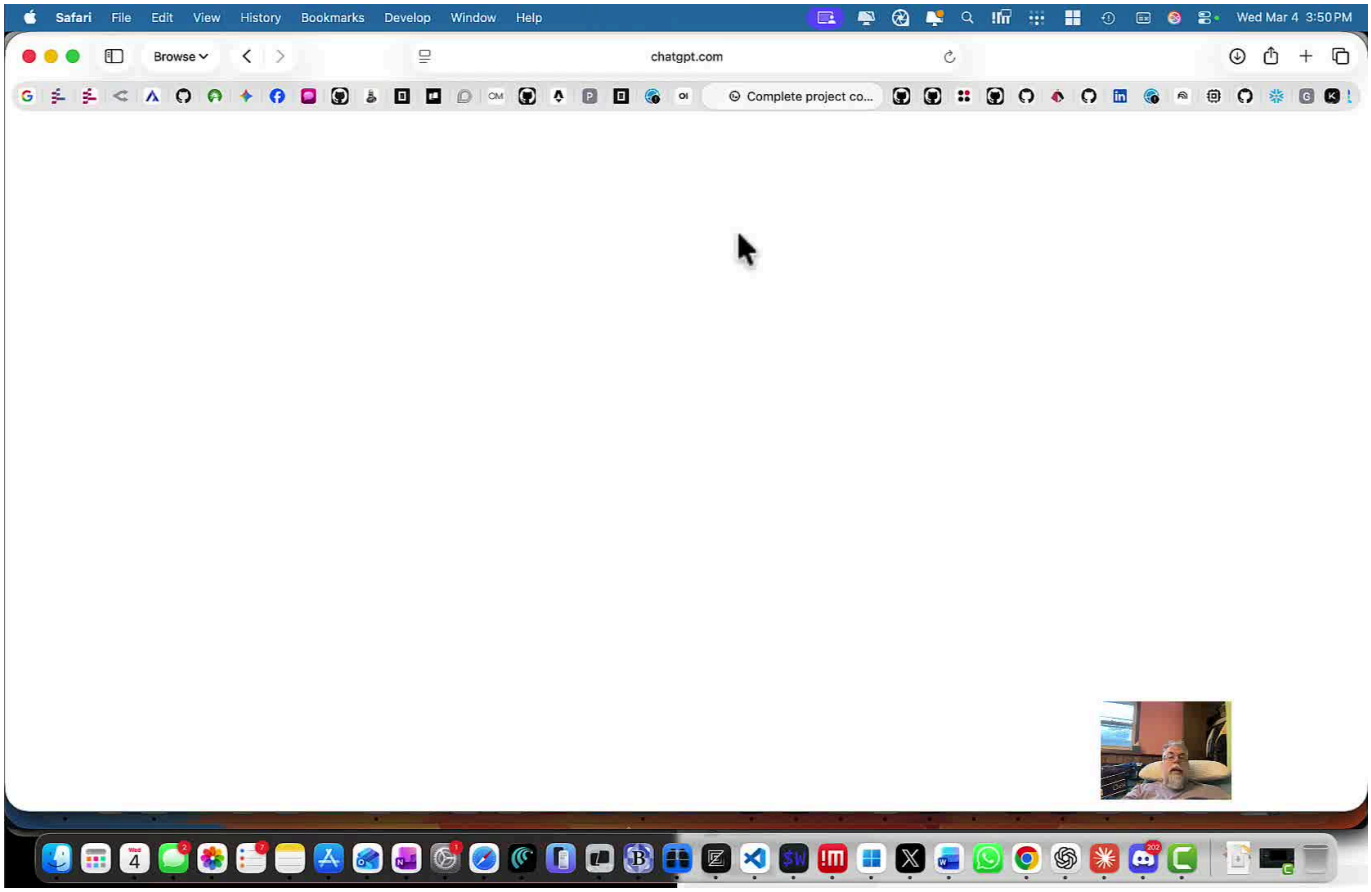
After the review, take the findings back into the workflow.

In the demonstrated process, review results are discussed with Codex, then turned into another sprint plan, and then fixed in another pass.

Use that same loop:

- review the findings
- decide what you agree with
- create follow-up sprint work
- apply fixes
- check the output again

The demonstrated example emphasizes a practical standard: the utility is good when it produces the needed files in a reliable, readable way.



Running and validating the application to fetch and process transcripts.

Run and validate the application

Run the completed app and inspect the output.

In the demonstrated workflow, this includes:

- asking how to run the app
- running it against the target channel
- checking the transcript output
- changing formatting so timestamps appear at the end of paragraphs
- running it again for a time range such as the past week

Validation here is straightforward: confirm that the app fetches transcripts, creates the intended files, and produces output in the format you actually want to use.

Back to top

Recap

You now have a grounded workflow for using Codex to build a transcript-processing utility:

- create a dedicated project folder
- use AI to define goals
- design the solution before coding
- break the work into sprints
- fetch YouTube transcripts
- improve naming and timestamp formatting
- run coding, testing, and review cycles
- validate the final output against your actual use case

If you continue from here, the most natural next step is to repeat the same pattern on a new transcript source or refine the utility further for scheduling, duplicate detection, or output formatting.

Back to top